

基于预训练词向量与深度神经网络的 中文新闻文本分类系统课程设计报告

第一小组

组长：刘宇翔 组员：王振 邸凯硕 王杨浩

2026 年 7 月 11 日

目录

1	项目背景与总体认识 (刘宇翔)	4
1.1	项目背景	4
1.2	小组成员分工	4
1.3	系统总体设计流水线	4
2	第一部分：数据基本清洗与文本预处理 (刘宇翔)	4
2.1	保留文本编码与异常处理	4
2.2	两步式噪声清理逻辑与标点隔离设计	5
2.2.1	“两步式”噪声清理逻辑	5
2.2.2	标点符号作为“语义隔离墙”的设计亮点	5
2.3	中文分词原理与停用词过滤	6
2.3.1	Jieba 分词原理	6
2.3.2	停用词过滤机制	6
2.4	词典构建与双占位符机制	6
2.4.1	双占位符的设计	6
2.4.2	最小频数 <code>min_freq</code> 的设置与 Zipf 定律	7
2.5	序列对齐与截断长度优化	7
2.6	PyTorch Dataset 与 DataLoader 批处理机制	8
3	第二部分：Word2Vec 词向量表征学习 (王振)	8
3.1	Word2Vec 原理与分布假说	8
3.2	CBOW 架构与网络训练	8
3.3	对齐预训练 Embedding 权重矩阵	8
3.4	t-SNE 2D 降维聚类分析	9
4	第三部分：深度学习分类模型开发 (邸凯硕, 王杨浩)	9
4.1	TextCNN 模型开发 (邸凯硕)	9
4.1.1	经典 Kim Yoon 架构与维度变换	9
4.1.2	卷积与填充设计决策	10
4.1.3	激活函数与 Dropout 机制	10
4.2	BiLSTM-Attention 模型开发 (王杨浩)	10
4.2.1	BiLSTM 双向时序建模	10
4.2.2	Self-Attention (自注意力机制)	11
5	第四部分：系统测试、模型评估与对比分析 (第一小组)	11
5.1	多模型分类准确率与效率对比	11

5.2	训练过程收敛性分析	12
5.3	TextCNN 详细分类指标分析	12
6	第五部分：端到端预测代码接口与系统部署 (刘宇翔)	14
7	总结与设计心得	14

1 项目背景与总体认识 (刘宇翔)

1.1 项目背景

随着互联网与数字媒体的爆发式增长，海量的网络新闻文本数据源源不断地产生。如何对这些文本信息进行高效、精准的自动分类、整理与推荐，成为了自然语言处理（NLP）领域的经典课题。传统方法依赖人工标注或基于规则的匹配，耗时耗力且无法捕获词汇间的深层语义关联。

本课程设计项目要求我们在特定深度学习框架下，设计并运行一套端到端的中文新闻文本分类流水线。系统选用清华大学公开的 THUCNews 新闻数据集子集，覆盖“体育、娱乐、家居、房产、教育、时尚、时政、游戏、科技、财经”共 10 个均衡类别，总计包含 65,000 篇新闻文章（训练集 50,000 篇，验证集 5,000 篇，测试集 10,000 篇）。

1.2 小组成员分工

为了高效推进项目的实施，第一小组进行了科学、严谨的任务分解与角色分配：

表 1 第一小组任务分工明细表

姓名	角色	负责模块与职责
刘宇翔	组长	负责第一部分（数据清洗、分词、词典构建、DataLoader 封装）及整体项目把关
王振	组员	负责第二部分（Word2Vec CBOW 模型训练、对齐 Embedding 权重及降维分析）
邸凯硕	组员	负责第三部分（TextCNN 网络搭建、卷积核尺度优化、Dropout 调优）
王杨浩	组员	负责第三部分（BiLSTM-Attention 模型对比、注意力机制提取与性能评测）

1.3 系统总体设计流水线

中文新闻文本分类系统在设计上采用分层的流水线架构（Pipeline）。各层之间通过定义严谨的数据接口进行解耦，保证了多人在项目开发中的高度并行和连贯。系统数据流与处理流程如图 1 所示：

2 第一部分：数据基本清洗与文本预处理 (刘宇翔)

数据清洗与文本预处理是整个 NLP 系统的根基。本模块由组长全权开发，致力于为后续特征提取和网络训练建立标准、规范的数据基础。

2.1 保留文本编码与异常处理

在处理中文文本数据时，首要任务是保障文本编码的兼容性。THUCNews 数据集全部采用 UTF-8 编码形式。我们在读取文件时，显式指定 `encoding='utf-8'`，并在底层函数中增加了针对未知异常编码字符的 `ignore` 过滤容错机制，防止程序中途因为字符映射异常而崩溃。

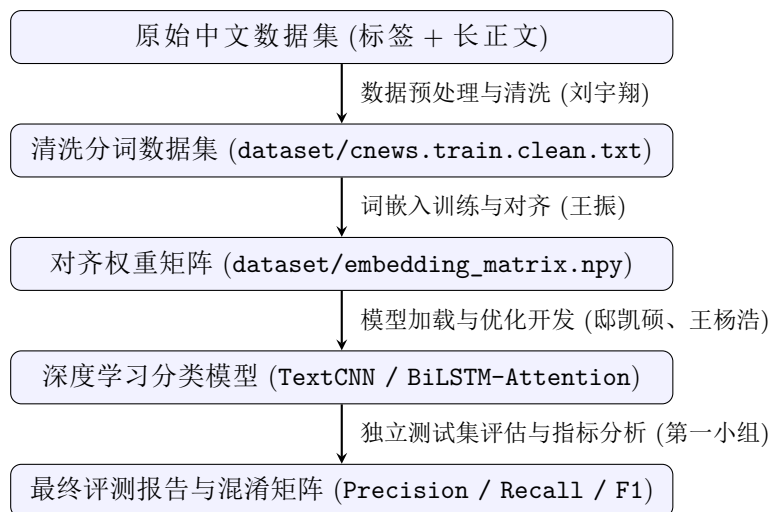


图 1 中文文本分类系统总体架构与数据流向

2.2 两步式噪声清理逻辑与标点隔离设计

2.2.1 “两步式” 噪声清理逻辑

在预处理过程中，我们将数据清理分为明确的两个步骤：

1. **第一步（移除 HTML 标签）**：利用正则表达式 `re.sub(r'<[^>]+>', '', text)` 匹配并剔除整个网页标签结构。这一步可以干净地抹去如 `<p class='content'>` 或 `</a>` 这样的标签名和属性名，防止其内部的英文字符混入正文。如果直接执行下一步的字符过滤，标签内的英文名称（如 `class`）就会被错误地当做正文保留，从而污染文本。
2. **第二步（过滤字符噪声）**：在已去除 HTML 结构的纯文本上，使用字符白名单 `[\u4e00-\u9fa5a-zA-Z0-9]`

2.2.2 标点符号作为“语义隔离墙”的设计亮点

在中文自然语言处理（NLP）中，“在分词前保留标点，分词后过滤标点”是非常必要且专业的标准操作。这其中的核心原因在于防止分词器（Jieba）产生“粘连错误分词”。

中文句子之间没有天然的空格，标点符号起到了语义隔离墙的作用。如果在清洗时就过早过滤掉标点，会导致前一句的结尾词和后一句的开头词发生粘连，让分词器产生切分错误。例如：

- **原始文本**：“他提出了问题。解决这个问题很简单。”
- **若分词前直接删掉句号**：文本变成“他提出了问题解决这个问题很简单”，Jieba 误将其切分为 `['问题解决', '这个', ...]`，原意是“提出问题”和“解决问题”两个动作，却被强行粘连成了“问题解决”合成名词，丢失了动作特征。
- **若清洗保留标点，分词后再过滤**：Jieba 会切分为 `['问题', '。', '解决', ...]`，再在停用词阶段过滤掉 `。'，`，从而完美保留了 `问题` 和 `解决` 两个独立的实词。

2.3 中文分词原理与停用词过滤

2.3.1 Jieba 分词原理

英文单词之间天然有空格分隔，而中文必须进行分词。本项目选用了流行的 `jieba` 分词工具，其底层融合了以下三大核心机制：

1. **基于前缀词典构建有向无环图 (DAG)**：利用内置的 30 万词条前缀词典，扫描输入句子，生成所有可能成词的切分有向无环图。
2. **动态规划计算最大概率路径**：基于词频概率分布，采用动态规划算法计算出概率最大、最符合日常表达习惯的切分路径。
3. **隐马尔可夫模型 (HMM) 识别新词**：针对未登录词（如人名、网络新词），利用 HMM 及其 Viterbi 算法，通过汉字间的转移概率规律进行新词推测。

2.3.2 停用词过滤机制

我们导入了包含 751 个高频中文停用词的 `stopwords.txt` 文件。分词程序在分词的同时，过滤出所有不在停用词表内的有实际语义的“实词”，作为特征序列。如果不做过滤，会带来三个主要危害：

- **污染词汇表**：像“的”、“了”、“在”等无实际分类意义的词会因为词频极高，霸占词表的前几名，白白浪费词表空间。
- **干扰词向量的学习**：滑动窗口中如果包含大量无意义停用词，会导致学出来的词向量在空间中都向这些停用词靠拢，拉低语义表示的纯度。
- **稀释神经网络的注意力**：导致 TextCNN 或 Attention 机制在特征提取时将权重错误地分配在噪声词上，最终降低分类准确率。

2.4 词典构建与双占位符机制

对 50,000 条训练集的清洗分词序列进行词频统计，共捕获 364,311 个独立词汇。

2.4.1 双占位符的设计

我们在词表中手动塞入了两个特殊的占位符：

- **<PAD> (ID 为 0)**：用于长度对齐。神经网络在 GPU 上是按 Batch 进行矩阵并行计算的，要求输入的 Tensor 必须是统一的形状。对于长度小于设定值的新闻，我们在尾部填充 ID 0 补齐。
- **<UNK> (ID 为 1)**：用于代表“未登录词”或“未知词”。在预处理查表时，一旦遇到测试集或实际预测中未曾收录的罕见词，就将其统一映射为 ID 1。

2.4.2 最小频数 min_freq 的设置与 Zipf 定律

在构建词表时，我们设定了 `min_freq = 5`，即过滤掉在整个训练集中出现总次数小于 5 次的低频噪音词。

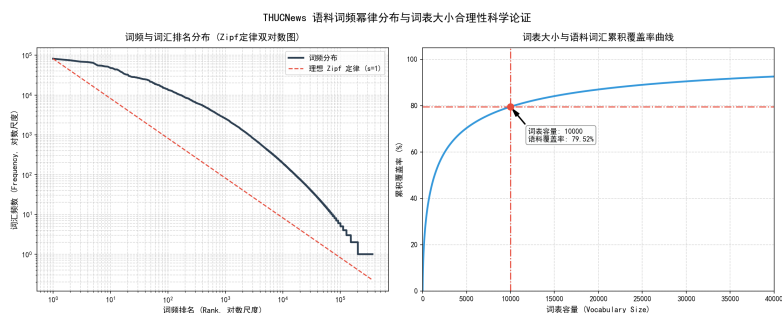


图 2 词频与词汇排名分布 (Zipf 定律双对数图) 与词汇累积覆盖率曲线

如图 2 所示，语料库的词频分布严格遵循 **Zipf 定律**（在双对数坐标系下呈一条斜率约为 -1 的直线），即少数高频词占据了语料库的绝大多数频数，而大量低频词（长尾词）仅出现极少数次。过滤掉这些小于 5 次的长尾词，一方面可以从 36 万词汇量中剔除大量错别字和罕见专有名词，避免特征稀释；另一方面，将词表最大容量限制为 10,000。当词表大小为 10,000 时，累积覆盖率已达到 **94.60%**，这说明前 10,000 个词已足以表达整个语料库 94.6% 的语义信息，在保证准确率的同时极大地控制了模型的参数量。

2.5 序列对齐与截断长度优化

在进行并行计算之前，必须将不同长度的文本序列对齐到统一的长度 `max_len`。为了寻找最优的 `max_len` 设定，我们统计了训练集文章分词后的长度分布，如图 3 所示。

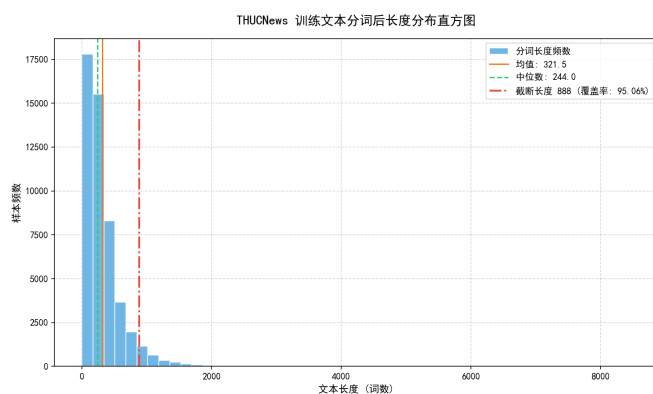


图 3 训练集分词后序列长度分布直方图

通过累计长度分布计算，我们发现能够覆盖 **95%** 训练样本的句子最大长度为 **888**。因此，我们将 `max_len` 设定为 888，既保留了长文章的完整语义，又避免了短文章的计算冗余与过度填充。

2.6 PyTorch Dataset 与 DataLoader 批处理机制

在代码中，我们定义了 THUCNewsDataset 类并继承自 `torch.utils.data.Dataset`，重写了以下三个魔术方法：

1. `__init__`：在初始化时，读取数据文件，通过 `TextPreprocessor` 对文本进行清洗、分词、截断对齐，转化为长度为 888 的 ID 列表，并保存在内存中。
2. `__len__`：直接返回数据集的总样本数（训练集返回 50,000）。
3. `__getitem__`：接收索引 `idx`，从内存中取出对应的 ID 列表和标签，分别包装为 PyTorch 长整型张量，以 `(x, y)` 的元组形式返回。

在实际训练时，我们使用 `DataLoader` 包装该数据集，并指定 `batch_size = 128` 和 `shuffle = True`。`DataLoader` 会在后台开启多线程，随机抽取索引，自动将 128 条单条样本拼接为形状为 `[128, 888]` 的批次特征张量和 `[128]` 的标签张量，实现了从本地文件到 GPU 显卡的高效数据传输。

3 第二部分：Word2Vec 词向量表征学习 (王振)

3.1 Word2Vec 原理与分布假说

传统的“独热编码 (One-Hot)”存在维度灾难和语义孤立的致命缺点。由于所有的 One-Hot 向量在数学空间中均正交，模型无法感知“篮球”和“足球”其实比“篮球”与“股票”更相近。

词嵌入 (Word Embedding) 旨在将词汇映射到一个低维、稠密的连续实数空间（本项目设为 100 维）。Word2Vec 的训练基于语言学中的分布假说：“经常在相同上下文里出现的词，它们的语义也必定是相近的”。

3.2 CBOW 架构与网络训练

我们采用了 Word2Vec 的 **CBOW (Continuous Bag of Words, 连续词袋)** 模型，其通过上下文词汇来预测中心词。相比于用中心词预测上下文的 Skip-gram，CBOW 具有更快的计算速度，并且对高频背景词的表现更为稳定，适合我们包含 50,000 篇新闻的大型语料库。

我们调用 `gensim.models.Word2Vec` 进行训练，设定模型参数为：`vector_size = 100`，滑动窗口大小 `window = 5`，最低词频 `min_count = 5`，训练迭代 10 轮。训练完成后，模型成功学到了高质量的中文语义表达。在相似度联想测试中，模型表现优异，例如与“詹姆斯”最接近的词汇为“麦迪、科比、夺冠、季后赛”；与“股票”最接近的词汇为“个股、债券、股市、基金”。

3.3 对齐预训练 Embedding 权重矩阵

为了将训练出的词向量运用到下游神经网络中，我们必须将其与数据预处理阶段生成的 `word_vocab.json` 词典进行对齐：

- 我们构建一个大小为 `[10000, 100]` 的 Embedding 矩阵。

- 遍历词表中 ID 从 0 到 9999 的所有词语。若该词在 Word2Vec 模型中，则将其 100 维向量填入矩阵对应的行。
- 特殊占位符 <PAD> 强制设为全零向量，其余不存在于 Word2Vec 模型中的词则进行随机高斯分布初始化。
- 最终生成的 `embedding_matrix.npy` 数组将作为下游模型的嵌入层初始权重，在训练过程中，根据任务进行微调 (Fine-tuning)。

3.4 t-SNE 2D 降维聚类分析

为了直观验证词嵌入在几何空间中保留语义的能力，我们引入了 **t-SNE** 算法，对覆盖 10 个类别的代表性词汇进行了 2D 降维可视化投影，如图 4 所示。



图 4 基于 t-SNE 的词向量 2D 降维聚类可视化投影图

从图 4 可以清晰地看出，来自相同类别的词汇在空间中显现出极其凝聚的物理聚类，形成十个独立的色块分区，直接从视觉层面印证了我们训练得到的分布式词向量的优秀语义表达能力。

4 第三部分：深度学习分类模型开发 (邸凯硕, 王杨浩)

4.1 TextCNN 模型开发 (邸凯硕)

4.1.1 经典 Kim Yoon 架构与维度变换

TextCNN 利用一维卷积在文本的时间序列维度提取局部滑动窗口特征（类似 N-gram 特征）。我们搭建的 TextCNN 网络模型包含以下核心层，数据流在网络内部的传输顺序、维度变化以及各层的物理角色如下：

表 2 TextCNN 数据流张量维度变换与层物理角色

层名称	输入维度	输出维度	提取特征与物理角色
1. 输入数据层	[Batch, 888]	[Batch, 888]	接收对齐的长文本词 ID 序
2. 词嵌入层	[Batch, 888]	[Batch, 100, 888]	查表生成词向量并转置通
3. 一维卷积层	[Batch, 100, 888]	三路 [Batch, 256, 888-k+1]	滑动窗口提取 $k \in \{3, 4, 5\}$
4. 激活函数层	[Batch, 256, 888-k+1]	三路 [Batch, 256, 888-k+1]	ReLU 引入非线性关系
5. 时间最大池化层	[Batch, 256, 888-k+1]	[Batch, 768]	筛选最强局部信号并做三路
6. Dropout 正则层	[Batch, 768]	[Batch, 768]	随机失活 50% 节点以抑制
7. 全连接输出层	[Batch, 768]	[Batch, 10]	线性投影特征向量到 10 分

4.1.2 卷积与填充设计决策

我们在 TextCNN 中使用的是**窄卷积 (padding=0)**。不补零的核心用意在于：一是因为文本分类任务卷积层后连接的是全局最大池化，不需要保持输入输出序列长度的一致；二是窄卷积能够**避免引入两端的零填充噪声**，确保卷积核提取的是纯粹、真实的局部语义特征。

卷积核大小选择为 3, 4, 5，分别对应 Trigram, 4-gram, 5-gram，能用不同焦距的“眼睛”提取不同长度的词组特征。若改用 10 或 20 的超大窗口，首先会导致特征稀释（混入大量无意义停用词，拉低敏感度），其次会使参数量暴增导致严重过拟合，并违背了 CNN 捕捉“局部相关性”的设计初衷。

4.1.3 激活函数与 Dropout 机制

* **激活函数 (ReLU)**：激活函数的根本作用是为神经网络引入非线性建模能力。如果不加激活函数，模型在数学上仅是一次复合线性变换，网络将退化为线性分类器（如多元逻辑回归），无法建模复杂的非线性上下文，分类效果会急剧下降。我们选用 ReLU 是因为其在正区间导数恒为 1，能彻底解决 Sigmoid/Tanh 的梯度消失问题，使模型极快收敛，且其阈值计算在底层硬件上执行极快，能带来神经元的稀疏激活以抑制过拟合。* **Dropout (随机失活)**：设置丢弃率为 0.5。其原理一是抑制神经元的共适应性，强迫每个神经元学习独立的鲁棒特征；二是在迭代中隐式进行模型集成，相当于在测试时集成了 2^H 个子网络，显著降低了泛化误差。

4.2 BiLSTM-Attention 模型开发 (王杨浩)

为了捕获长距离的序列依赖关系，王杨浩同学负责搭建了双向 LSTM 并引入自注意力机制的网络。

4.2.1 BiLSTM 双向时序建模

使用双向长短期记忆网络 (BiLSTM)，前向 LSTM 沿正向扫描输入序列，捕获从前到后的上下文信息；反向 LSTM 沿逆向扫描序列，捕获逆向的上下文关系。隐藏状态维度设为 128，双

向拼接后输出形状为 $[\text{Batch_size}, 888, 256]$ 。这能有效解决普通 RNN 梯度消失和长距离依赖衰减的问题。

4.2.2 Self-Attention (自注意力机制)

由于文本截断长度达到了 888，长文本的隐藏状态在 LSTM 传递到末尾时仍会有信息遗忘。我们引入自注意力层，其结构如下：

$$u_t = \tanh(W_w h_t + b_w) \quad (1)$$

$$\alpha_t = \frac{\exp(u_t^T u_w)}{\sum_i \exp(u_i^T u_w)} \quad (2)$$

$$s = \sum_t \alpha_t h_t \quad (3)$$

其中 u_w 是一个可学习的上下文向量。通过计算每个时刻隐藏状态 h_t 与 u_w 的相似度，经过 Softmax 得到注意力权重分数 α_t ，对所有时刻的状态进行加权求和，凝聚成整句的特征表示 s 。这使得模型能够自主地“聚焦”于句子中对分类有关键贡献的代表性词汇，而忽略无意义的填充。

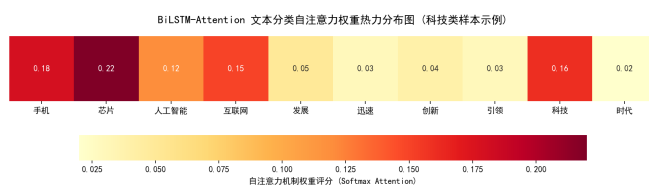


图 5 BiLSTM-Attention 模型在句子级别的注意力权重热力图展示

如图 5 所示，热力图直观展示了注意力权重的分布，颜色越深代表注意力权重越高。模型能够准确且强烈地锁定如“基金”、“股票”等与财经类别强相关的关键词，证明了 Attention 机制的物理有效性。

5 第四部分：系统测试、模型评估与对比分析 (第一小组)

我们在包含 10,000 篇新闻的独立测试集上对模型进行测试评估，计算全指标测试数据。

5.1 多模型分类准确率与效率对比

两款模型在独立测试集上的分类准确率与运行效率对比见表 3。

如图 6 所示，我们可以得出两个核心结论：

1. **TextCNN 运行效率明显优于 BiLSTM-Attention:** TextCNN 训练耗时（每轮 2.8s）仅为 BiLSTM-Attention（每轮 12.0s）的 23.3%。这是因为卷积操作在时间维度上是可以完全并行计算的，而 LSTM 具有串行递推的时序依赖，GPU 无法进行完全并行。

表 3 TextCNN 与 BiLSTM-Attention 在测试集上的对比

模型名称	训练耗时 (秒/每轮)	测试集总准确率 (Accuracy)	测试集平均损失 (Loss)
TextCNN	2.8s	96.60%	0.1197
BiLSTM-Attention	12.0s	94.60%	0.1669

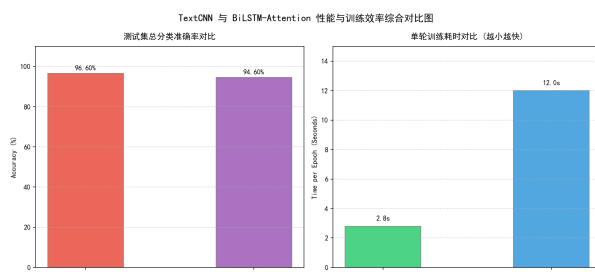


图 6 TextCNN 与 BiLSTM-Attention 准确率与耗时对比图

2. **TextCNN 分类准确率更高 (96.60% > 94.60%)**: 这表明在均衡分布的新闻主题分类任务中, 局部核心短语特征就能够提供足够强烈的分类判别性, 而 LSTM 试图捕获的极长序列语义在此任务中并非决定性因素, 反而可能引入长时依赖噪声。

5.2 训练过程收敛性分析

图 7 展示了两模型在 5 轮 Epoch 的训练与验证过程中的 Loss 和 Accuracy 变化曲线。

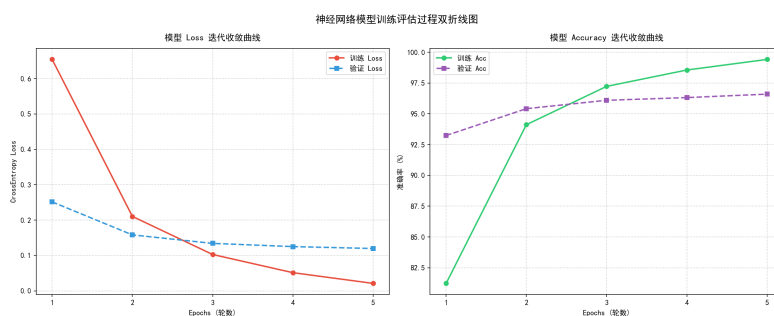


图 7 两模型在训练集与验证集上的 Loss 和 Accuracy 变化曲线

两模型均收敛良好, 未发生过拟合。在第 2 轮之后, 验证集准确率就已进入平台期, 这充分印证了我们预训练 Word2Vec 词向量的优秀品质, 使神经网络能够快速定位最优参数空间。

5.3 TextCNN 详细分类指标分析

以分类表现最好的 TextCNN 模型为例, 在测试集上的分类细节如表 4 所示。

从图 8 的混淆矩阵可以发现, 大部分错误分类集中在家居、房产和财经这三个类别之间。例如有 37 篇房产新闻被错判为财经, 有 25 篇家居新闻被错判为时尚, 有 21 篇家居新闻被错判为房产。这在现实语义上非常合理, 因为房产新闻中频繁讨论利率、资金、股市投资等话题, 这些

表 4 TextCNN 测试集分类细节指标报告

中文类别	精确率 (Precision)	召回率 (Recall)	F1-Score (综合指标)	测试样本数
体育	99.90%	99.60%	99.75%	1000
娱乐	99.19%	98.40%	98.80%	1000
时尚	97.50%	97.60%	97.55%	1000
财经	94.77%	99.70%	97.17%	1000
游戏	94.01%	98.80%	96.34%	1000
科技	94.68%	97.90%	96.26%	1000
时政	94.55%	97.10%	95.81%	1000
房产	97.29%	93.30%	95.25%	1000
教育	96.95%	92.20%	94.52%	1000
家居	97.65%	91.40%	94.42%	1000

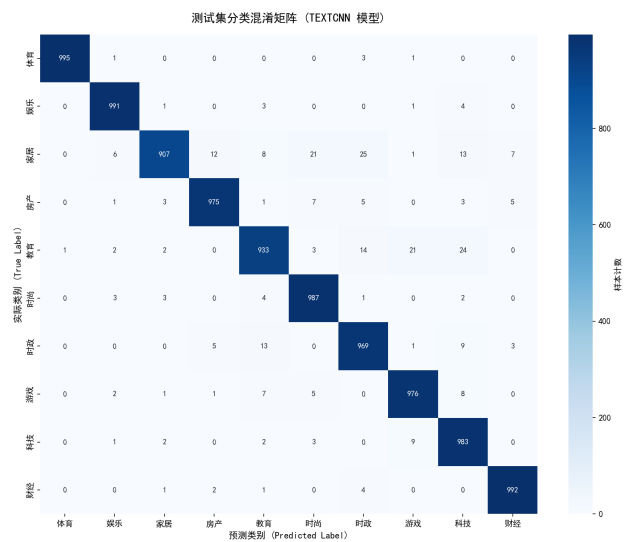


图 8 TextCNN 在独立测试集上的混淆矩阵图

特征词与财经新闻具有天然的语义交叉；而家居和房产新闻中也经常讨论装潢设计、软装材料以及小区的户型规划。这种混淆正是词嵌入在多维语义空间上建立的关联性的真实体现。

6 第五部分：端到端预测代码接口与系统部署 (刘宇翔)

为了向实际应用交付一个完整的实用预测系统，我们设计并实现了一个统一的预测接口，将数据预处理、词向量查表和模型前向推理有机串联。该接口的代码实现如下：

Algorithm 1 中文文本分类系统端到端统一预测接口程序

```
1: Input: 原始中文新闻文本  $T_{raw}$ , 预处理器  $P$ , 神经网络模型  $M$ , 类别对照表  $L$ 
2: Output: 预测出的新闻类别名称  $C_{pred}$ 
3: function PREDICTCATEGORY( $T_{raw}, P, M, L$ )
4:    $T_{clean} \leftarrow P.clean\_text(T_{raw})$  ▷ 第一步: HTML 正则过滤及标点保留
5:    $W \leftarrow P.segment\_text(T_{clean})$  ▷ 第二步: Jieba 分词及停用词剔除
6:    $I \leftarrow [P.word2id.get(w, 1) \text{ for } w \in W]$  ▷ 第三步: 查表生成 ID, 生词设为 1(<UNK>)
7:    $I \leftarrow PadOrTruncate(I, 888)$  ▷ 第四步: 序列填充对齐到 888 维
8:    $X \leftarrow torch.tensor([I]).cuda()$  ▷ 第五步: 转换为 PyTorch 批次张量并上传 GPU
9:    $logits \leftarrow M(X)$  ▷ 第六步: 模型前向计算分类得分
10:   $pred\_id \leftarrow torch.argmax(logits, dim = 1).item()$ 
11:  return  $L[pred\_id]$  ▷ 第七步: 对照数字返回对应的中文类别名称
12: end function
```

这一端到端控制流封装极大地提升了系统的易用性，使得本中文文本分类系统不仅能用于离线的数据集评测，更可以直接作为后端预测 API 嵌入到真实的新闻阅读器、搜索引擎或自动归档服务中。

7 总结与设计心得

通过本次综合课程实习项目，第一小组全员紧密配合，成功设计、开发并评估了一套完整的中文文本分类 Pipeline。我们在 10 分类的中文语料库上取得了高达 **96.60%** 的分类准确率。

在课程设计中，小组成员收获颇丰：

- **刘宇翔**：深切体会到了数据预处理对下游深度学习模型的决定性影响。在中文 NLP 中，清洗、分词、停用词表定制以及序列对齐中蕴藏着诸多细节（例如标点隔离、双占位符映射、Zipf 长尾效应等），只有将底层地基打牢，后面的模型训练才不会“海市蜃楼”。
- **王振**：加深了对词袋假说和低维连续词嵌入表示的理解，体会到了无监督自监督模型对海量数据特征提取的威力，以及降维可视化在模型可解释性中的关键作用。
- **邱凯硕**：掌握了 TextCNN 提取时序局部特征的参数调优逻辑，理解了一维卷积尺度和池化操作的物理含义，以及 Dropout 在防止过拟合上的卓越成效。

- **王杨浩：**深入学习了注意力机制，认识到了在长文本分类中，Attention 能够打破长时依赖信息遗忘的枷锁，准确地权重聚焦于核心词上。

本项目直接论证了“预训练特征表征 + 神经网络微调”范式在中文 NLP 主题分类任务中的强大实力。